Lexical Analysis Phase for Durin's Code

Overview

Lexical analysis is the very first phase of the Durin's Code compiler. Its job is to read the raw source file character by character and transform that stream of text into a flat, structured sequence of **tokens** — the smallest meaningful units of the language. Think of it as the compiler's "reading" phase: before it can understand the grammar of a program, it must first recognize its vocabulary. The output of this phase is a token stream that feeds directly into the parser in Phase 2.

What the Lexer Must Read

Looking at the Durin's Code syntax from your proposal, the source file contains several distinct categories of text that the lexer needs to identify and classify:

- Reserved keywords like `room`, `action`, `item`, `npc`, `exit`, `if`, `else`, `print`, `remove`, `true`, `false`
- Player-specific dotted accessors like `player.inventory`, `player.health`, `player.has_item`, `player.win`
- String literals wrapped in double quotes like `"bag_end"`, `"take ring"`, `"The Precious is yours."`
- Identifiers (user-defined names) like `the_ring`, `sauron`, `buckland`, `mount_doom`
- Property keys and their values like `power: 100`, `health: 999`, `hostile: true`
- Operators and comparators like `==`, `&&`, `+=`, `=`
- Structural punctuation: `{`, `}`, `(`, `)`, `:`
- Directional keywords used in exits: `north`, `south`, `east`, `west`
- Integer literals like `100`, `999`
- Comment lines beginning with `//`
- Whitespace and newlines (typically discarded but used to separate tokens)

Token Types (The Token Taxonomy)

The lexer needs a well-defined catalogue of token types. Each character sequence it recognizes gets assigned one of these types along with its **lexeme** (the actual text) and its **source position** (line and column number for error reporting). Here's the full classification:

1. Keywords

These are reserved words that have fixed meaning in Durin's Code and cannot be used as identifiers. The lexer must check every identifier-like string against a keyword lookup table. The keyword set includes:

`room`, `item`, `npc`, `exit`, `action`, `if`, `else`, `print`, `remove`, `player`, `true`, `false`, `and` (if supported textually), `has_item`, `win`

2. Directional Literals

Exit directions are a special sub-category of keywords specific to the world-building syntax: `north`, `south`, `east`, `west`. They look like identifiers but carry semantic meaning in exit declarations, so the lexer can either treat them as keywords or let the parser distinguish them — but pre-classifying them makes the parser's job cleaner.

3. Identifiers

Any sequence of letters, digits, and underscores that begins with a letter and does not match a reserved keyword. Examples from your proposal: `the_ring`, `sauron`, `bag_end`, `buckland`, `mount_doom`. These represent user-defined room names, item names, and NPC names.

4. String Literals

Any sequence of characters enclosed in double quotes. The lexer must handle the opening `"`, consume all characters until the closing `"`, and emit the content *between* the quotes as the lexeme. This must also handle edge cases like escaped characters (`\"` inside a string) and unterminated strings (a string that never closes before end-of-file — a lexical error). Examples: `"bag_end"`, `"The air is thick with ash."`, `"destroy ring"`.

5. Integer Literals

Sequences of one or more digit characters (`0–9`). The lexer captures them as a raw string and optionally converts them to an integer value. Examples: `100`, `999`, `0`.

6. Operators

Single or multi-character operator symbols. The lexer must use **maximal munch** — always consuming the longest possible match — so `+=` is one token, not `+` followed by `=`, and `==` is one token, not two `=` signs. The operator set includes:

Token Type	Lexeme
ASSIGN	=
PLUS_ASSIGN	+=
EQUALS	==
AND	&&
DOT	.
COLON	:

7. Punctuation / Delimiters

These are structural characters that define blocks and groupings:

Token Type	Lexeme
LBRACE	{
RBRACE	}
LPAREN	(
RPAREN	)

8. Comments

Lines beginning with `//` are comments. The lexer should recognize the `//` sequence and then consume everything until the end of that line, emitting **no token at all**. Comments are silently discarded but the line counter must still be incremented so that error messages reference correct line numbers.

9. Whitespace & Newlines

Spaces, tabs, and newlines carry no semantic meaning in Durin's Code (it is brace-delimited, not indentation-sensitive). The lexer skips them but must increment its internal line counter every time it encounters a newline character (`\n`) so that source positions remain accurate.

10. EOF (End of File)

When the lexer exhausts the input, it emits a special `EOF` token. This signals to the parser that the token stream has ended and allows clean termination.

The Lexer's Internal Mechanics

The Input Buffer

The lexer maintains a **read pointer** (or cursor) into the source text. At each step it looks at the current character (often called `peek` or `lookahead`) to decide which path to take. Some tokens require looking one character ahead without consuming it — this is called **single character lookahead** and is standard in most lexers.

The Finite Automaton Model

Conceptually, the lexer is a **Deterministic Finite Automaton (DFA)**. Each token type corresponds to a regular expression, and the DFA transitions between states as it reads characters. For Durin's Code:

- Upon seeing a letter or underscore → enter the *identifier/keyword* state, keep consuming alphanumeric characters and underscores until a non-identifier character is hit, then check the result against the keyword table
- Upon seeing a digit → enter the *integer literal* state, keep consuming digits
- Upon seeing `"` → enter the *string literal* state, consume until closing `"` or error on newline/EOF
- Upon seeing `/` → peek at the next character; if it's also `/`, enter the *comment* state and consume the rest of the line; otherwise emit an error (since bare `/` is not a valid token in this language)
- Upon seeing `+` → peek ahead; if the next character is `=`, consume it and emit `PLUS_ASSIGN`; otherwise emit an error
- Upon seeing `=` → peek ahead; if the next character is also `=`, consume it and emit `EQUALS`; otherwise emit `ASSIGN`
- Upon seeing `&` → peek ahead; if the next character is also `&`, emit `AND`; otherwise emit a lexical error

- Upon seeing `{`, `}`, `(`, `)`, `:`, `.` → emit the corresponding single-character token immediately

The Symbol/Keyword Table

The lexer maintains a **static hash map** of all reserved words. After recognizing a sequence of characters as a potential identifier, it performs a lookup in this table. If the lexeme is found, the token type is set to the corresponding keyword type (e.g., `KW_ROOM`, `KW_ACTION`, `KW_IF`). If not found, it is classified as a plain `IDENTIFIER`. This design keeps the DFA simple — it doesn't need separate states for every keyword.

Token Data Structure

Every token the lexer emits should be a structured record containing at minimum:

- **type** — the token category (from the taxonomy above)
- **lexeme** — the exact character sequence from the source
- **line** — the line number in the source file
- **column** — the character position on that line

The line and column fields are critical for producing useful error messages in later phases. For example, if the semantic analyser finds that an exit references a non-existent room, it can point back to the exact token's source position.

Lexical Errors Specific to Durin's Code

The lexer is the first line of defence for error reporting. Errors it must catch include:

- **Unterminated string literal** — a `"` is opened but the line ends or the file ends before a closing `"` is found. Error message should report the line where the string began.
- **Invalid character** — a character that belongs to no valid token pattern, such as `@`, `#`, `$`, or `%`. The lexer should report it with its exact position and attempt to recover by skipping it and continuing.
- **Malformed operator** — a single `&` with no second `&` following it, or a single `|` (if not in the language). The lexer should flag this before the parser ever sees it.
- **Numeric identifiers** — an identifier starting with a digit (e.g., `3ring`) is not valid. Since the lexer enters the number path upon seeing `3`, it will emit `INTEGER(3)` and then `IDENTIFIER(ring)` — which the parser will reject. This is technically a parser error, but good lexer design can detect and report the anomaly more clearly.

Handling the Dotted Player Accessors

One subtlety in Durin's Code is the `player.inventory`, `player.health`, `player.win`, and `player.has_item(...)` syntax. The lexer has two reasonable strategies:

Option A — Emit as separate tokens: Emit `KW_PLAYER`, then `DOT`, then `IDENTIFIER(inventory)`. The parser is then responsible for recognising the dotted-access pattern. This is the cleaner, more general approach and is recommended because it keeps the lexer simple and lets the parser enforce the grammar.

Option B — Emit as compound tokens: Treat `player.inventory` as a single `PLAYER_ATTR` token. This simplifies some parser rules but makes the lexer more complex and less flexible.

Option A is preferable for a university compiler project as it maintains a clean separation of concerns between phases.

Output of the Lexical Analysis Phase

Running the lexer on the example snippet `room "bag_end" { exit east "buckland" }` would produce a token stream like:

```
KW_ROOM       "room"       line 1, col 1
STRING_LIT    "bag_end"    line 1, col 6
LBRACE        "{"          line 1, col 16
KW_EXIT       "exit"       line 2, col 3
KW_EAST       "east"       line 2, col 8
STRING_LIT    "buckland"   line 2, col 13
RBRACE        "}"          line 3, col 1
```

This flat, ordered token stream — free of whitespace and comments — is what the **parser** in Phase 2 will consume to build the Abstract Syntax Tree.

Summary

The lexical analyser for Durin's Code is responsible for transforming raw source text into a clean, typed token stream. It must correctly classify keywords, identifiers, string and integer literals, operators (including multi-character ones), structural punctuation, and directional words,

while silently discarding whitespace and comments. It serves as the primary source of **line and column tracking** for all downstream error reporting, and it performs the first layer of error detection by catching unterminated strings, invalid characters, and malformed operators before the source ever reaches the parser.